Local RAG Application

Ask questions about your own documents - fully offline, no external APIs

Author	Sai Venkat
Repository	github.com/saivenkat2322/RAG-application
Stack	Flask, FAISS, sentence-transformers, Ollama (Llama 3.2)
Platform	Windows, Python 3.12, VS Code

1. Overview

This project is a Retrieval-Augmented Generation (RAG) web application that runs entirely on a local machine. A user uploads PDF documents through a browser interface and then asks questions in natural language. The app finds the passages of the documents most relevant to the question and passes them to a locally hosted large language model, which writes an answer grounded in that content.

Because the language model is served locally by Ollama and the vector search runs in-process with FAISS, the application needs no cloud account, API key, or internet connection after setup, and uploaded documents never leave the user's computer.

Key goals

- Answer questions from the user's own documents instead of relying only on the model's general knowledge.
- Keep all data and computation local for privacy and zero API cost.
- Provide a simple REST API and a lightweight web UI that a friend or recruiter can run in a few commands.

2. Technology Stack

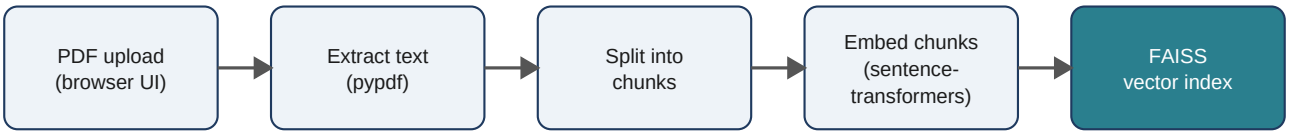
Component	Technology	Role
Web backend	Flask 3.0.3, Flask-CORS 4.0.1	REST API and serving the frontend
Document parsing	pypdf 4.3.1	Extracts text from uploaded PDFs
Embeddings	sentence-transformers 3.0.1	Converts text chunks and questions into vectors
Vector store	faiss-cpu 1.14.3, NumPy 1.26.4	Similarity search over chunk embeddings
Language model	Ollama running Llama 3.2	Generates the final answer from retrieved context
HTTP client	requests 2.32.3	Calls the local Ollama server from the backend
Frontend	HTML page (frontend/index.html)	Upload and chat interface in the browser

3. Architecture

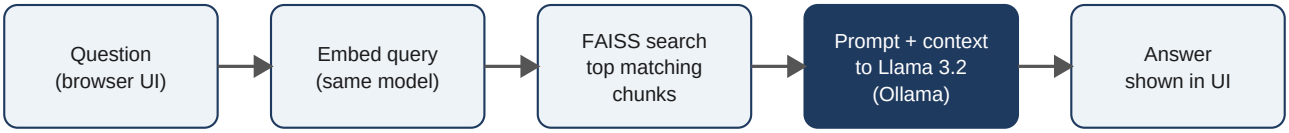
The application has two main flows. When a document is uploaded, its text is extracted, split into chunks, embedded as vectors, and stored in a FAISS index. When a question is asked, the question is embedded

with the same model, FAISS returns the closest chunks, and those chunks are placed into a prompt for Llama 3.2 so the answer is based on the uploaded material.

A. Indexing flow (document upload)



B. Query flow (asking a question)



Flask REST API connects the browser UI to the RAG engine. Everything runs on the local machine.

Why this design

- **Local-first:** Ollama and FAISS both run on the user's machine, so there are no usage fees or data leaving the device.
- **Separation of concerns:** `app.py` handles HTTP routes; `rag.py` holds the retrieval and generation logic.
- **Simple deployment:** a single Python process plus the Ollama service is all that is needed.

4. Project Structure

```
rag-clean/
|-- app.py            Flask app and API routes
|-- rag.py            RAG engine: indexing, retrieval, answer generation
|-- frontend/
|   `-- index.html    Browser UI for upload and chat
|-- uploads/          Uploaded documents (contents ignored by Git)
|-- requirements.txt  Pinned Python dependencies
|-- README.md         Setup instructions
`-- .gitignore        Excludes venv, caches, uploads
```

5. REST API

The backend exposes three endpoints on port 5000. The browser UI calls them automatically, and they can also be tested with any HTTP client.

Method	Endpoint	Purpose
GET	/api/health	Health check; the UI calls it to confirm the backend is running.
POST	/api/upload	Accepts a document, extracts its text, and adds it to the vector index.

Method	Endpoint	Purpose
POST	/api/chat	Accepts a JSON body with a `query` field and returns the generated answer. An empty query returns HTTP 400.

Example chat request

```
POST /api/chat
Content-Type: application/json

{ "query": "What are the main points of the document?" }
```

6. Setup and Running

Requirements

- Python 3.12 (faiss-cpu does not yet support Python 3.14).
- Ollama installed from ollama.com and running in the background.
- About 8 GB of RAM or more for the Llama 3.2 3B model.

Installation steps (Windows PowerShell)

```
git clone https://github.com/saivenkat2322/RAG-application.git
cd RAG-application
py -3.12 -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
ollama pull llama3.2
python app.py
```

Then open `http://127.0.0.1:5000` in a browser, upload a PDF, and start asking questions. The first run downloads the sentence-transformers embedding model, so an internet connection is needed once.

7. Challenges and Solutions

Problem	Cause	Solution
faiss-cpu failed to install	The default Python on the machine was 3.14, which faiss-cpu does not support yet.	Installed Python 3.12 and created the virtual environment with the `py` launcher (`py -3.12 -m venv venv`).
Frontend did not load correctly	A path mismatch between the Flask app and the location of the frontend files in `app.py`.	Corrected the frontend path so Flask serves `frontend/index.html`; verified end to end in VS Code.
Git push returned 403	Windows had saved GitHub credentials for a different account than the repository owner.	Removed the stored credentials with `cmdkey` and re-authenticated as the correct account.

8. Security and Privacy Notes

- **Local processing:** documents, embeddings, and model inference all stay on the user's machine.
- **Debug mode disabled:** the app runs with `debug=False` and binds to `127.0.0.1`, so it is not reachable from other devices on the network.
- **Private data kept out of Git:** `.gitignore` excludes the virtual environment, Python caches, `.env` files, and uploaded documents.

- **Development server:** Flask's built-in server is meant for local use. A production deployment would need a WSGI server and authentication.

9. Sharing the Project

The code is published at `github.com/saivenkat2322/RAG-application`. GitHub stores the source only; it does not run the app. Anyone who wants to use it clones the repository and follows the setup steps above on their own machine, so each person runs their own private copy with their own documents.

10. Possible Improvements

- Containerize the app with Docker so setup is a single command.
- Show source citations (document and passage) alongside each answer.
- Support more file types such as DOCX, TXT, and Markdown.
- Persist the FAISS index to disk so documents do not need to be re-uploaded after a restart.
- Add authentication and deploy to a cloud VM with a production WSGI server.
- Add automated tests for the API routes and retrieval quality.